



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF ALGORITHMS AND THEIR APPLICATIONS

Terraforming Mars in OpenGL

Supervisor:

Csongor Csanád Karikó

Doctoral Student

Author:

Norbert Varga

Computer Science BSc

Budapest, 2025

Contents

I	Introduction	3
II	User documentation	5
1	Hardware and software requirements	6
1.1	Hardware	6
1.2	Software	6
2	Installation	6
3	Usage of the application	6
3.1	Menu	7
3.2	Game	8
3.2.1	States	8
3.2.2	Elements	12
3.3	Debug tools	17
3.3.1	Menu	17
3.3.2	Game	18
III	Developer documentation	19
1	Developer environment	20
1.1	Utilized libraries	20
1.2	Compilation	20
1.3	Runtime environment	21
2	Specification	21
3	Architecture	22
3.1	Model	24
3.1.1	The board	25
3.1.2	Decks and cards	29
3.1.3	<i>::model</i>	34

3.2	View	41
3.2.1	Utilities	42
3.2.2	Views	45
3.2.3	Animations	47
3.2.4	Rendering	48
4	Testing	49
 IV Conclusion		50
 Acknowledgements		51
 A Simulation results		52
 Bibliography		53
 List of Figures		53
 List of Tables		55
 List of Algorithms		56
 List of Codes		57

Part I

Introduction

Terraforming Mars is a board game for 1 to 5 players. In it, the players control corporations which race to terraform the planet Mars. To achieve this, they must raise the temperature, form oceans and add oxygen to the atmosphere, and alongside these they create settlements, plant forest and much more.

The gameplay consist of generations, and each generation has three phases: research phase, action phase and production phase. In the research phase, players can choose from a few cards that represent projects contributing to the taming of Mars. In the action phase, players can use these cards, use actions on specific played cards, or perform standard projects. In the production phase, players get to replenish their resources based on their production values. Mars is represented by a hexagonal grid, on which players can place oceans, greeneries and cities.

The goal of this thesis was to implement this game with the ability to easily expand it: there are a variety of board variants, expansions, decks which the players can choose from. The other goal was to develop the application in a high performance, highly customizable environment supporting 3D rendering and models. Therefore, C++ and OpenGL were chosen for this project.

Part II

User documentation

1 Hardware and software requirements

1.1 Hardware

- 64-bit CPU with the x86 instruction set
- GPU supporting OpenGL 4.6
- 1 GB RAM
- mouse, optionally keyboard

1.2 Software

- Operating system: Windows 11
- GPU driver supporting OpenGL 4.6

2 Installation

The usage of the software does not require any form of setup or installation. After extracting the archive, the application can be run with the *view.exe* executable.

3 Usage of the application

The user can quit the application at any point by clicking the *Close* button of the window, or by pressing the following keyboard combination: **Alt + F4**

Fullscreen mode can be toggled at any point by pressing the key **F11** key.

3.1 Menu



Figure 1: The menu screen of the application with an ongoing game.

The menu screen has the following three buttons the user can interact with:

- **Continue:** this button is only shown if a game has already been started. Pressing this button brings back the player to the game screen with the ongoing game.
- **New Game:** pressing this button creates a new game and transitions the app to the game screen. If there was an ongoing game, it is lost forever.
- **Quit:** pressing this button exits the application, as if the *above detailed* methods were used. If there was an ongoing game, it is lost forever.

3.2 Game

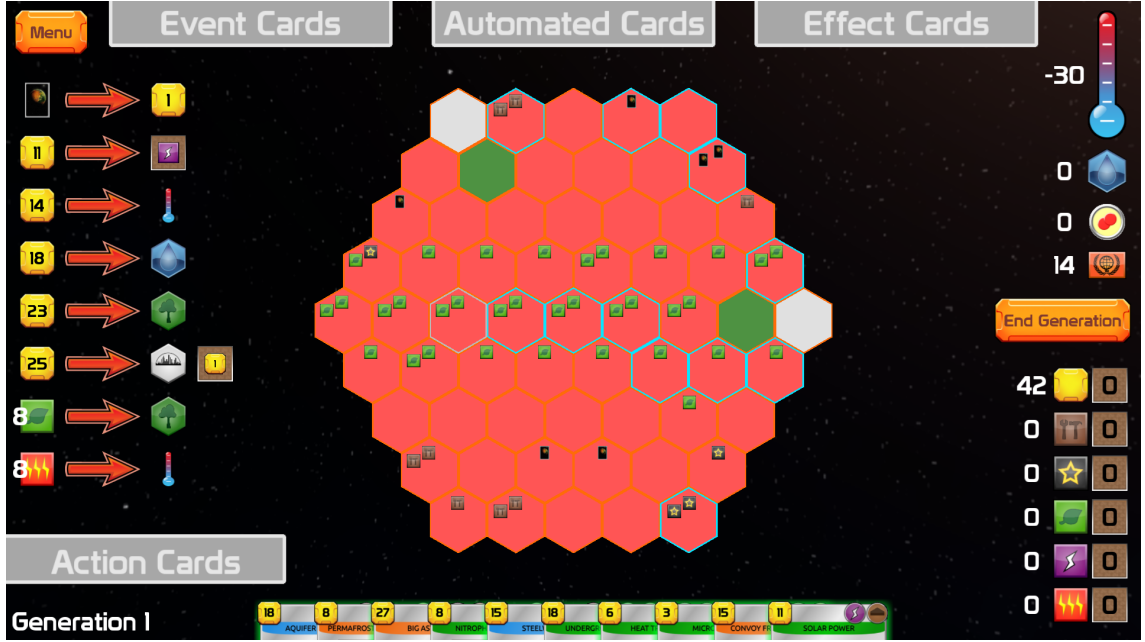


Figure 2: Game screen in the action state.

The player should be familiar with the *game rules*, in particular with the additions of the solo variant. Furthermore, these modifications are in effect:

- The player always controls the beginner corporation (10 cards for free and 42 Credits).
- Only about half of the cards from the basic set are included.

3.2.1 States

The game can be in different so called '*states*'. A state just means that certain actions may or may not be available to the player. The states are:

- **Action state:** This state is almost the same as the *Action phase* of the board game. The player can play cards from their hand, use standard projects or convert resources, use actions on their active cards and end the generation with the *End button*. The slight difference here is when using the *Sell patents* standard project, the game enters the *Sell state* where they can actually sell cards in their hand.
- **Sell state:** When using the *Sell patents* standard project, the game enters this state. In this state only the following actions can be performed:

-
- All cards in the *hand* glow red. The player has the option to drag out any card from their hand and play it. Playing the card sells it, discarding the card and giving one Credit to the player. For more detail about dragging out and playing cards, check the description of the *hand*.
 - The label of the *End button* is replaced. Clicking on it exits the *Sell state*, going back to the *Action state*.
- **Payment confirmation state:** When playing a card or using an action that can be bought with resources, the game will enter this state. In this state every action is disabled, and a window is shown where the player can confirm the amount of the resources they wish to spend. Confirming the payment advances the game, and returns to the *Action state*.



Figure 3: Payment confirmation window.

- **Placement confirmation state:** When playing a card or using a standard project or conversion that places a tile, the game will enter this state. The border of the tiles on *Mars* that are valid will be colored white, and the others will be dark. In this state every action is disabled, but the tiles of *Mars* are clickable, and clicking on a valid one places the tile there. A hint text informs the player what kind of tile they are placing currently. Confirming the placement advances the game, and returns to the *Action state*.



Figure 4: Placement confirmation. On the top, the tile to be placed is shown, and valid locations have a white border.

- **Research state:** In the *Action state*, clicking on the *End button* automatically performs the production phase, and after that the game will enter this state. In this state four cards will appear on the screen. The player needs to decide how many and which they want to purchase. Initially all of them are set for purchasing, and the player can toggle them between purchasing and discarding. This is backed up with visual cues. A hint text informs the player how much they are going to play for the selected cards. The label of the *End button* is replaced. Clicking on it confirms the research, and the game returns to the *Action state*.

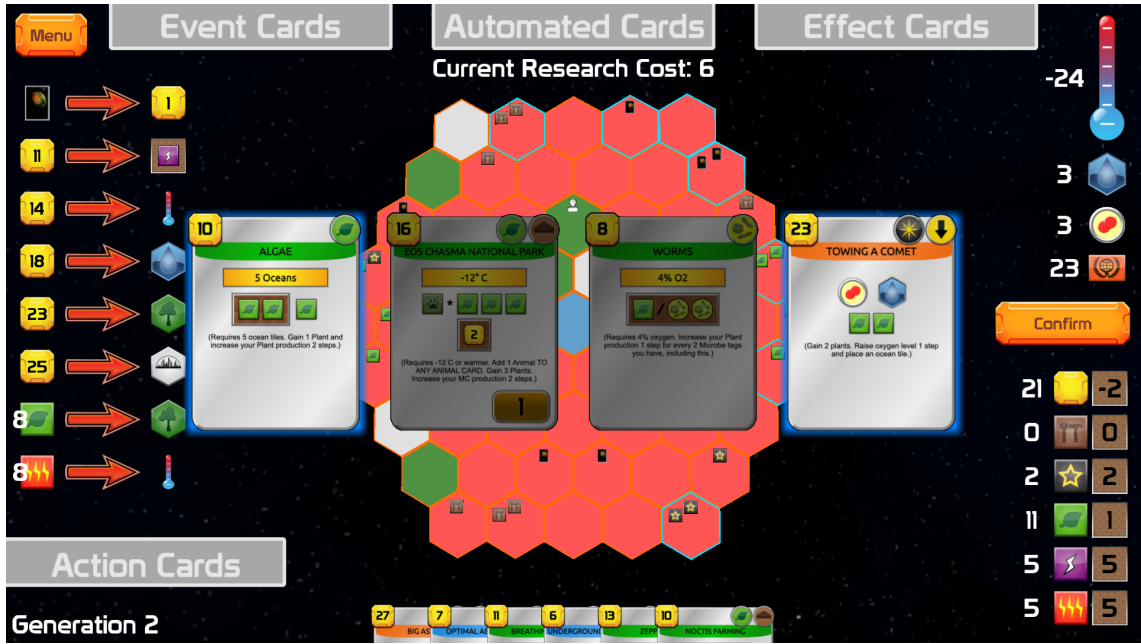


Figure 5: Research phase.

- **Post last generation state:** After ending the last generation and performing production phase, if the player has enough Plant resources to convert them to a greenery, the game will enter this state. Here, only the *Convert Plants to greenery* standard project can be used and the *End button* to finish the game.
- **Game over state:** After the last generation the game will enter this state. The game will display if the player has won or not, and the amount of victory points they have earned. Here, the player's only option is returning to the menu.



Figure 6: End of a game.

3.2.2 Elements

The game screen consists of the following visual elements:

- **Menu button:** Located in the top left corner. This button can always be clicked, and takes the player back to the menu screen. The ongoing game gets paused, and can be resumed by selecting the *Continue button* on the menu screen.
- **End button:** Located on the right side. It can have multiple behaviors, depending on the *state*.
 - In the *Action state*, the end button ends the generation.
 - In the *Sell state*, the end button exits the *Sell state*.
 - In the *Research state*, the end button confirms the research.
- **Standard projects and converting resources:** Located on the left side of the screen under the *Menu button*. When they can be used, hovering over them gives the visual cue of enlarging the row of the action. From top to bottom:
 - **Sell patents:** The application enters the *Sell state*.
 - **Power Plant:** For 11 Credits, you increase your Energy production one step.

-
- **Asteroid:** For 14 Credits, you raise temperature one step.
 - **Aquifer:** For 18 Credits, you place one ocean tile.
 - **Greenery:** For 23 Credits, you place one greenery tile.
 - **City:** For 25 Credits, you place one city tile and increase your Credit production one step.
 - **Convert Plants to greenery:** For 8 Plant resources, you place one greenery tile.
 - **Convert Heat to temperature:** For 8 Heat resources, you raise temperature one step.

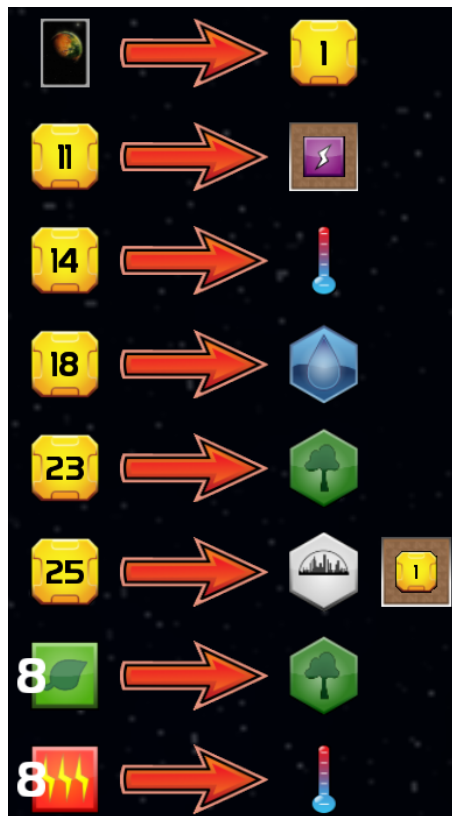


Figure 7: Standard projects and resource conversions.

- **Generation counter:** Located in the bottom left corner. Displays the current generation.
- **Global parameters:** Located on the upper right side of the screen. Shown parameters in order are: temperature, ocean count, oxygen, terraform rating. The three global parameter's indicators turn green when they have reached their respective goal.



Figure 8: Global parameters and terraform rating.

- **Resources:** Located on the lower right side of the screen. Consists of three columns: the first is the amount of resources, the second specifies the resource type and the third is the production output of said resource. The resources from top to bottom are as follows: Credit, Steel, Titanium, Plant, Energy, Heat. Values cannot be reduced below 0, except for Credit production which can be at a minimum of -5.



Figure 9: Resources. Notice how Credit generation is the only one that can go into negative, down to -5.

- **Hand:** Located at the bottom of the screen. Cards are by default hoverable: moving the mouse over a card enlarges it, showing the full card.

Some *states* allow the cards to be 'played'. In this case when a card is hovered, it can be dragged around with the mouse by pressing and holding the left mouse button. Two things can happen when the player releases the card: if the position of the mouse (and card) is above an invisible line, the card will get 'played', which is defined by the *state*. Otherwise, it will go back to the player's hand. There is always a visual indicator showing which one is about to happen.

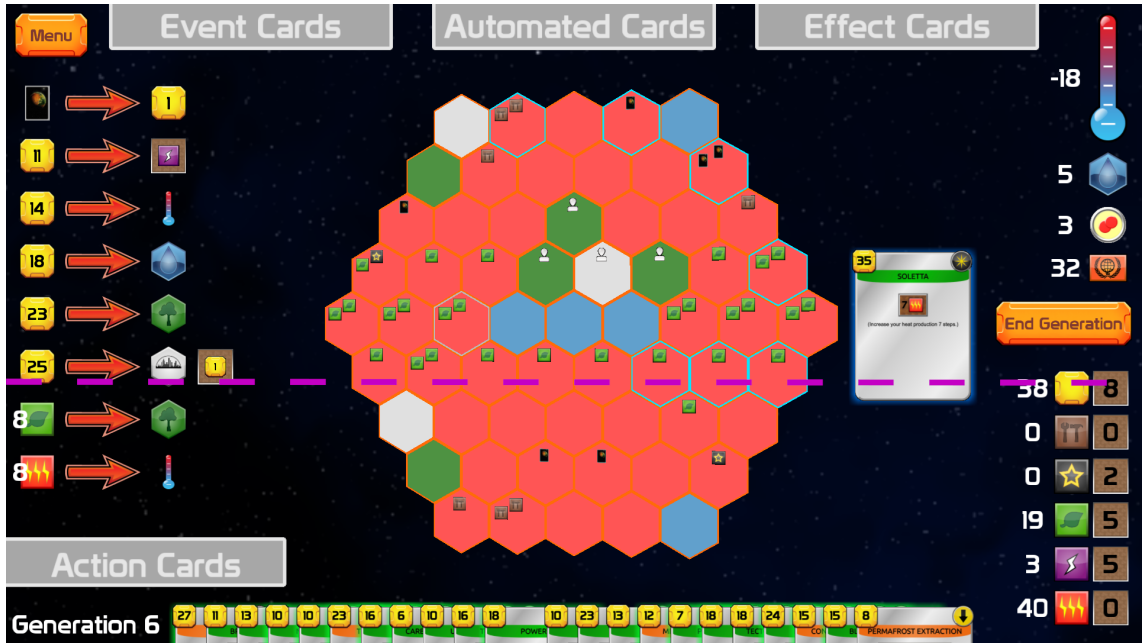


Figure 10: Approximate location of the above-mentioned invisible line.

- In the *Action state*, all cards that can be played will have a green glowing effect. If the mouse is over the invisible line, the card will have a blue glowing effect. The player can drag non-playable cards, but going over the invisible line automatically lets go of the dragged card.
- In the *Sell state*, all cards will have a red glowing effect. If the mouse is over the invisible line, the card will have a faded out effect. 'Playing' a card will sell it.



(a) Green highlight, usually indicating that the card can be played.



(b) Blue highlight, usually indicating that the card can be used or will be played.



(c) Red highlight, indicating that the card can be sold.



(d) Faded out card, usually indicating that the card cannot be used, will not be bought or will be sold.

- **Played card panels:** Four panels are located on the screen: action cards on the bottom left, event cards, automated cards and effect cards on the top. The panels open when they get clicked on, and close when any part of their body gets clicked, except the cards. They can also be toggled with the following keyboard keys: **U**, **I**, **O** and **P**. These can be opened at most times, but they do get closed automatically at certain events.

By default, only 8 cards can fit on a panel. If there are more out of the view, an arrow will be shown, which can be used to step one page in that direction. This can also be done with the **Left** and **Right** keys.

The action panel has further functionality: the card's actions can be used from here. If a card can be used, it will have a blue glowing effect. In this case, clicking on it will use the action of the card. If it was already used in the

generation, it will have a faded out effect. If it was not used in the generation but can't be used, it won't have any effects.

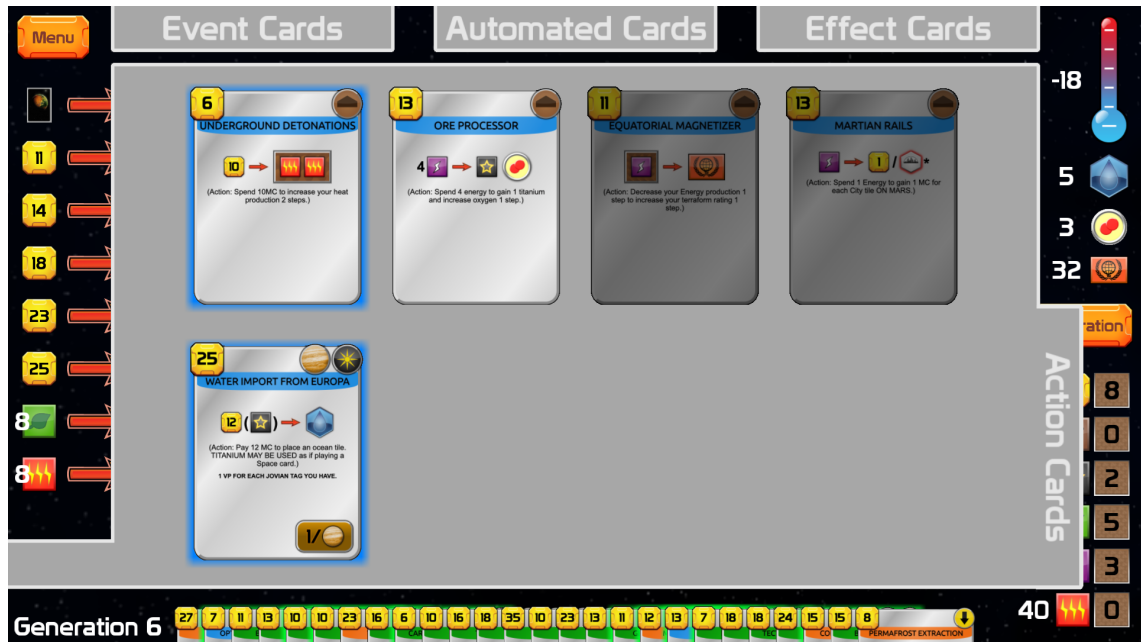


Figure 12: Action card panel. *Equatorial Magnetizer* and *Martian Rails* have been used this generation. While the others are still usable, *Ore Processor* cannot be used at this moment (the player does not have the required cost of 4 Energy).

- **Mars** Located in the center of the board. Empty tiles show their respective placement bonuses. Tiles owned by the player will have a player icon. Tiles reserved for oceans have a blue border. How placing tiles works is further explained under the section *Placement confirmation state*.

3.3 Debug tools

There are a few debug tools built in to the application. Both screens have a debug window that can be toggled by pressing **Ctrl + D**.

3.3.1 Menu

The debug window allows the user to set a seed for their game. Starting two games with the same seed means the random tiles will generate to the same place, and cards will come in the same sequence.

move this section to developer documentation

3.3.2 Game

The debug window allows the user to give themselves resources, resource production, raise their terraform rating, draw cards, raise temperature, raise oxygen and place ocean, greenery or city tiles.

Warning! The debug tool should only be used in the Action State. Usage in other states may cause the application to crash.

Pressing **F3** toggles the basic camera and the editorial camera. With the editorial camera, the user can use look around with the mouse and fly around with the following keys: **A**, **W**, **S**, **D**, **E**, **Q**¹.

¹A - left, W - forward, S - back, D - right, E - up, Q - down

Part III

Developer documentation

1 Developer environment

The program uses the **C++20** major version. Libraries were installed using the **vcpkg** package manager.

I developed the application on a laptop with **Windows 11**. I used Microsoft's IDE, **Visual Studio 2022** utilizing the **Desktop development with C++** workload and the **GLSL language integration** extension. It used version **10.0.22621.0** of the **Windows SDK**.

1.1 Utilized libraries

The following libraries and versions have been used for the application. Use this command to install the required dependencies using vcpkg: `.\vcpkg install sdl3 sdl3-image[jpeg,png] glew glm imgui[opengl3-binding,sdl3-binding] freetype -triplet x64-windows`

- **OpenGL:** (Apache License 2.0) - graphics rendering library.
- **OpenGL Extension Wrangler library (GLEW):** (MIT License) - library providing an interface to OpenGL.
- **OpenGL Mathematics:** (MIT License) - math functions, vectors, matrices, transformations, easing functions and more.
- **Simple DirectMedia Layer (SDL):** (zlib License) - window and input manager.
- **Dear ImGui:** (MIT License) - graphical user interface for debugging and tooling.
- **FreeType:** (The FreeType Project License) - library for font rendering.

1.2 Compilation

The project needs to be compiled with the following macros being defined: `SDL_MAIN_HANDLED`, `GLM_ENABLE_EXPERIMENTAL`, `GLM_FORCE_SWIZZLE`.

1.3 Runtime environment

Compiling the program in *Release* mode assembles the program in the *x64/Release/* directory. *view.exe* expects the assets, fonts and shaders to be in their respective directories²: Visual Studio 2022 should take care of this during compilation. This directory can now be archived or moved, as it contains all the files required for running the application.

2 Specification

Menu

The player should be able to start a new game and exit the application. Furthermore, if there is an ongoing game, the user should be able to continue said game.

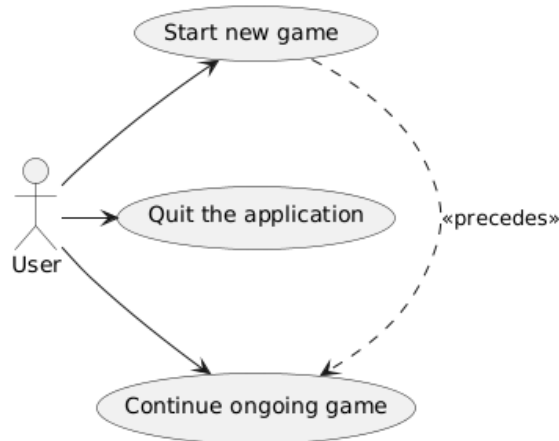


Figure 13: Use case diagram of the menu screen.

Game

The player should always have the option to leave the game. Available use cases change based on the *state* of the game.

²*assets/, fonts/, shaders/*

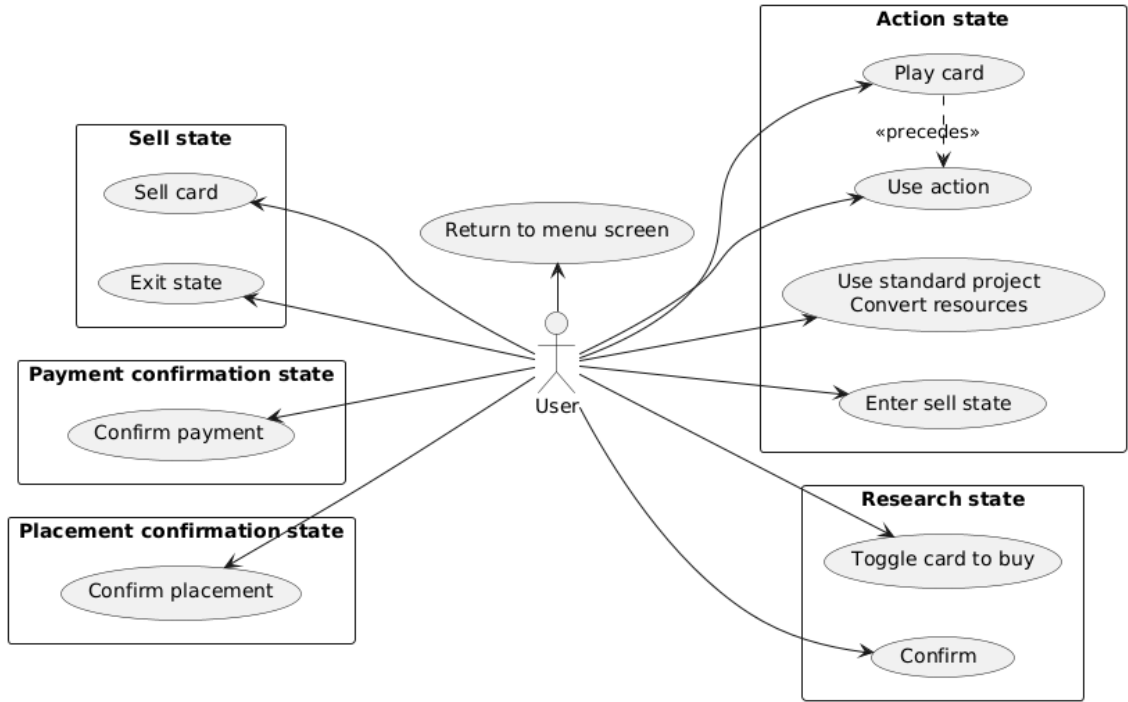


Figure 14: Use case diagram of the game screen.

Increasing a global parameter always increases the player's terraform rating.

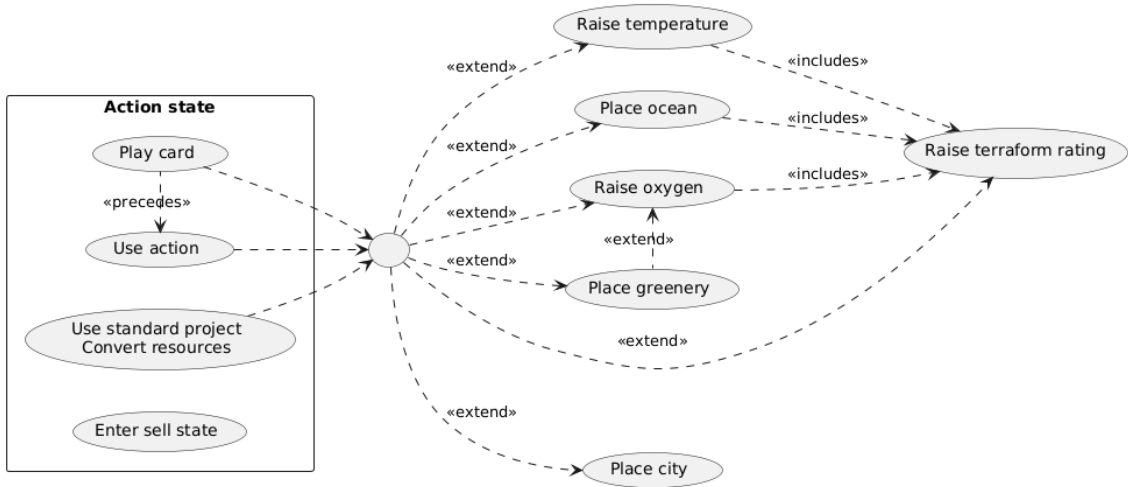


Figure 15: Detailed use cases of the Action state.

3 Architecture

The application has two layers: *Model* and *View*. Business logic is completely separated in the *Model*, and is platform independent. This means that separate frontends could be developed for this one backend.

The main aspect during development was extensibility: there are multiple boards that can be played on, different decks using different cards, extensions which add new cards, new resources, or even new game features. In this documentation you can find multiple explanations for implementing new varieties.

Since the game utilizes random chance (e.g. for card drawing), another aspect was that the application could be seeded, so that given the same seed for the game, the same random events happen.

View manages and renders the window, stores information for rendering and communicates with the *Model*. My implementation used OpenGL for rendering.

The **App** class holds together the application: it stores *View* instances, and injects instances of the *Model* into the corresponding *View* instance.

main.cpp is the entry point for the application. It manages the following responsibilities:

- Initializing submodules and libraries.
- Instantiate and initialize an **App** instance.
- Run the *render loop*. Here, it handles window events and forwards them to **App**, such as mouse and keyboard input. **App**'s **Update** and **Render** methods are called here.
- Clean up once the application is about to exit.

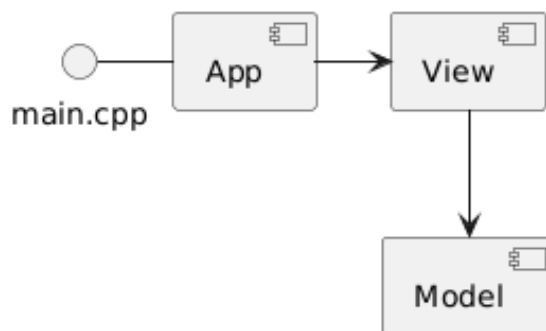


Figure 16: Package diagram of the application

Each file is self-contained, and only includes headers from the standard library other than the project. All enumerated values are *enum classes*.

There are some classes that have circular dependencies, for example **GameModel** and **GameModelState**. This was made possible with the following steps:

-
- Create *forward declaration headers* for the classes. This header should only have the declaration of it's class in it. Notice the naming of the file.

```
1 namespace model
2 {
3     class GameModel;
4 }
```

Code 1: game_model.fwd.hpp

- In the header files, the first lines should be including every **.fwd.hpp* file that is involved in the circular dependency. Include the rest of the header files as normal. DO NOT include the normal header file of other classes in the circular dependency.
- In the **.cpp* files, include the normal headers of all the classes needed.

This structure solves these circular dependencies, however, there are a few drawbacks:

- Since in the header files only the declaration of the other classes is present, no function calls can be made here. This means that inline functions cannot use the other classes.
- From the previous point, it is apparent that classes participating cannot be template classes, as these require all their definitions in their respective header files.

There is some leeway though: one header file from the two can include the other one. In this case, the structure of files and includes should still be upheld.

3.1 Model

Every code in this project is under the namespace *::model*.

To avoid magic numbers in the code, I have created a *constants.hpp* header file, which consists of constants defined with *inline constexpr*. This can be included from any other file - even from *View*.

3.1.1 The board

The `::model::boards` namespace contains the part of the code that implements the game board. Mars is made up of hexes, and to be more specific, the hexes have pointy-top orientation. Storing tables made up of hexes is not trivial, and there are

forrásmegjelölés

multiple ways to implement it.

I choose to go with the axial coordinate system. Here every square has a q and an r coordinate. Increasing the q coordinate moves right horizontally, and increasing the r coordinate moves down, but instead of vertically, it moves on the line you get by rotating the q axis 60° in the negative direction. This alignment matches with the usual layout in programming, where the y -axis is directed downwards.

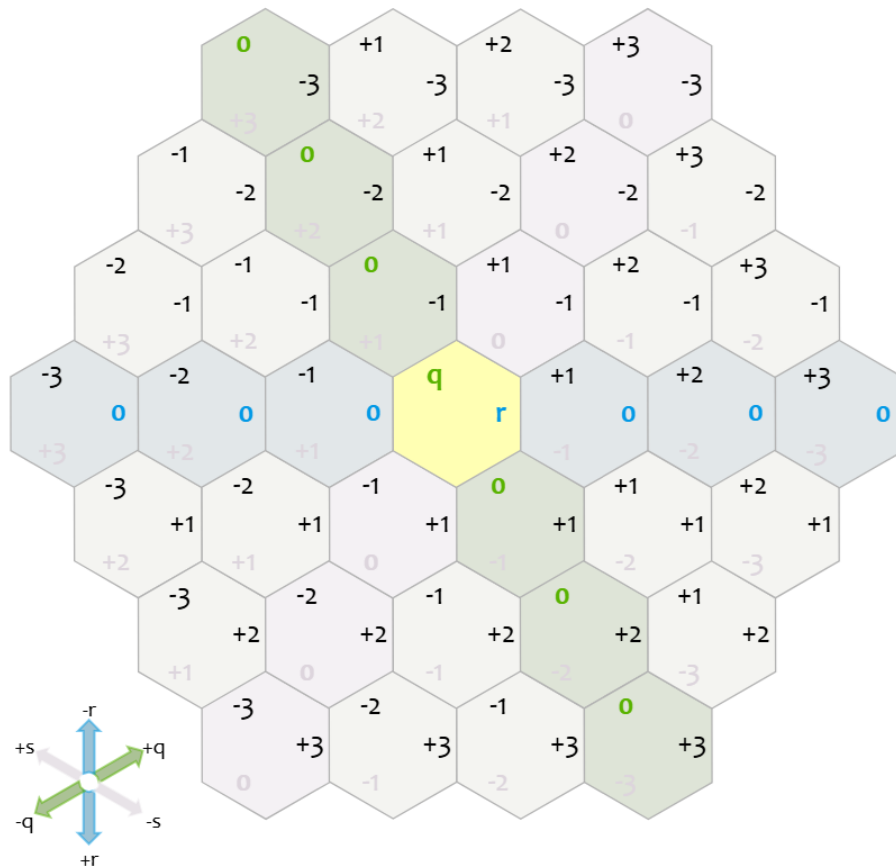


Figure 17: The axial coordinate system. Notice the third coordinate in a lighter color.

It is important to note, that the axial coordinates are derived from cube coordinates, which have a third coordinate, s . But since the equation $q + r + s = 0$ is true for all tiles, we can identify each square with just two coordinates, and calculate s if we need it.

ezt itt ki kell
jobban fejteni,
vagy elég hogy
az osztálydia-
grammon rajta
van?

The `::model::boards` namespace has one enumerated data type, `TileType`.

Tile

This class represents a single tile on the board. For convenience, they store their q and r positions, as well as their current type and owner. Oceans cannot be owned, but an empty tile can have an owner. Placement bonuses are also stored in each class and can be applied to the placer.

Boards

To implement different boards in the application, I have decided to use the *bridge design pattern*, as

- it hides the underlying data structure in the form of the implementor,
- it enables different board layouts and implementations, as classes can derive from the implementor, each one representing a different board variant, *but* at the same time
- it makes the boards extensible, as classes can derive from the abstraction and implement functionality specific to them. Since we are deriving from the abstraction, all concrete implementors automatically gain this extended functionality.

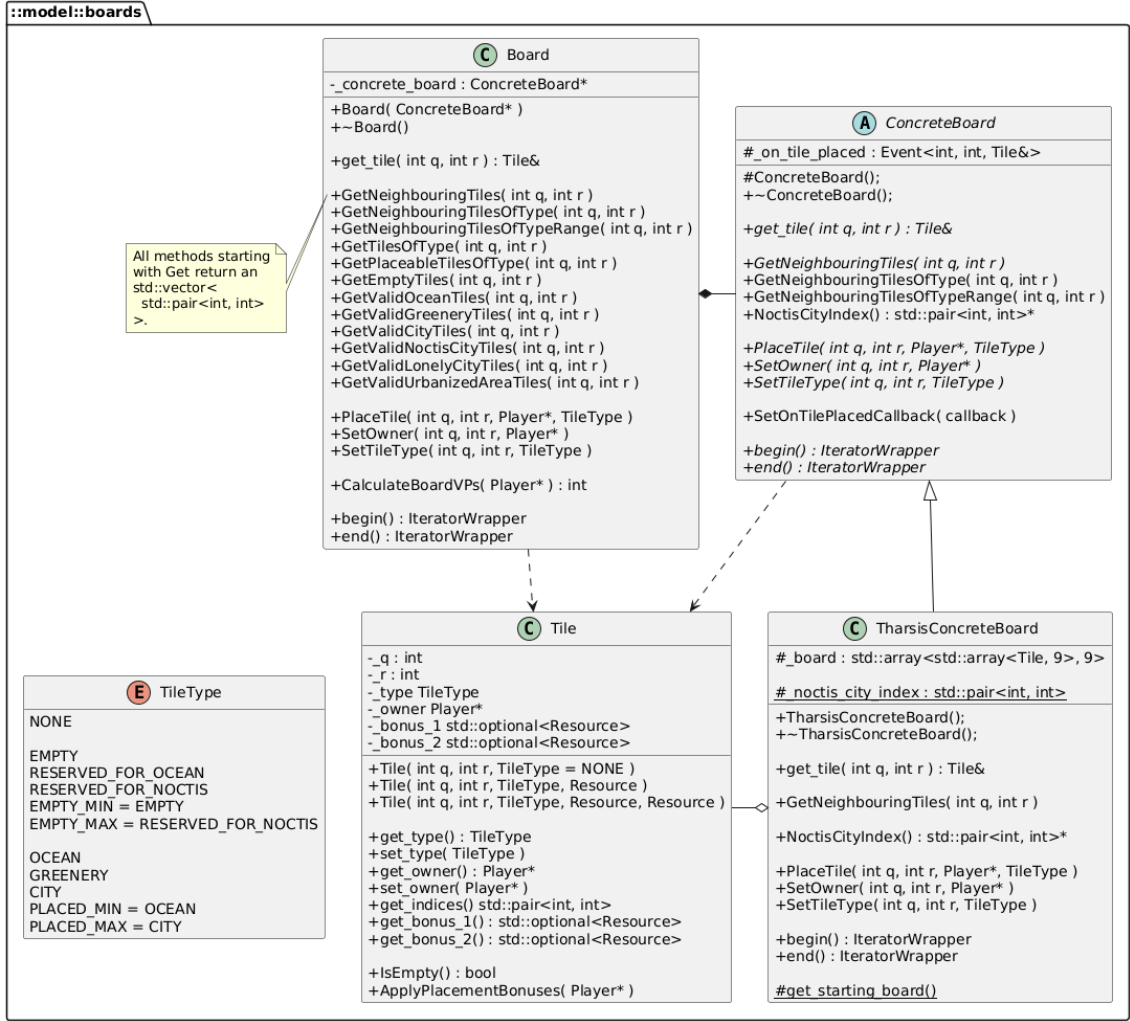


Figure 18: Class diagram of the `::model::boards` namespace.

As Figure 18 shows, **ConcreteBoard**'s methods are almost all virtual or abstract. Board stores an instance of **ConcreteBoard**, and extends the interface of it with methods required for the game. A new board can simply be implemented by inheriting from **ConcreteBoard**. Here there are no such classes, but Board can also be derived from, to implement additional behavior, for example some functionality specific to an extension.

Since the game board as a whole is also hex shaped, the tiles do not make out a full rectangle. There are multiple solutions to storing a board, and each and every subclass of **ConcreteBoard** can implement their own.

For **TharsisConcreteBoard**, I choose the 2D array implementation shown on Figure 19. I made this decision because indexing is easy and fast compared to other solutions and the elements don't move, they stay in memory for a full game, so after the initial allocation only fields of the elements are modified. This also means that

the unused memory is negligible. Furthermore, special off-board tiles can be stored on the empty ones.

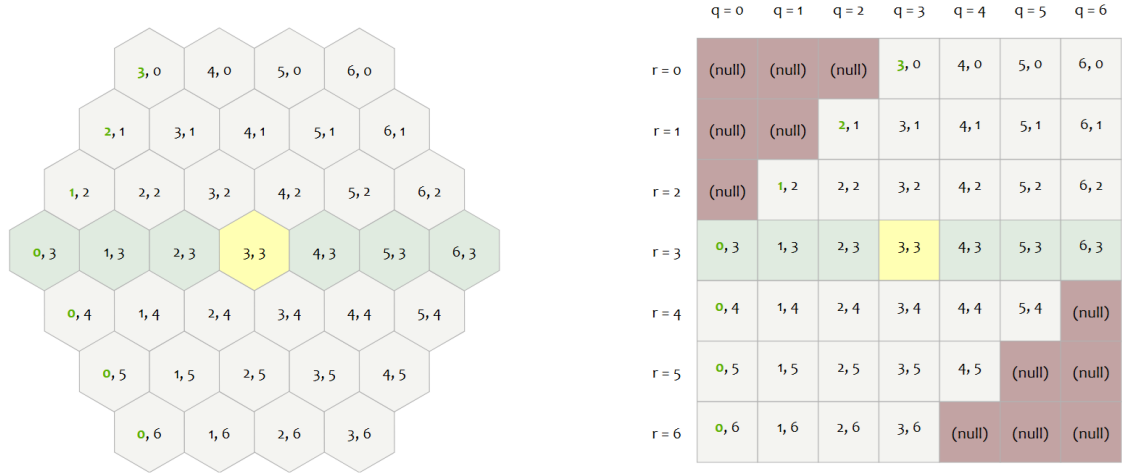


Figure 19: Example of memory layout when storing a board in a 2D array.

Iterators

Notice the `begin` and `end` methods in the class diagram on Figure 18. It was a requirement that boards could be iterated over. However, there were multiple obstacles I needed to overcome during the design and implementation of a universally usable iterator.

Since there could be any number of different board implementations, each one could potentially have a fundamentally different iterator. This sparked the interest to make an abstract **Iterator** base class, and derive iterators for every **ConcreteBoard** implementation.

But having **Iterator** as a base class meant that `begin` and `end` cannot simply return an **Iterator** instance because of object slicing³. The problem is, returning a pointer to an **Iterator** created by `new` does not sound good either: nobody expects a pointer when they call `begin` on a container and no one expects that they need to call `delete` on it. This would just be bad design. Furthermore, it would be a pointer: the semantics for using the **iterator** would be different, and you could not even use it for a range-based for loop.

My solution: I created a child class of **Iterator** called **IteratorWrapper**. It has a constructor that takes an **Iterator** pointer - any one of our concrete iterators -

³The derived class needs more memory than its parent class.

automatically deletes it when **IteratorWrapper** goes out of scope, and delegates every operation to the **Iterator** it is holding.

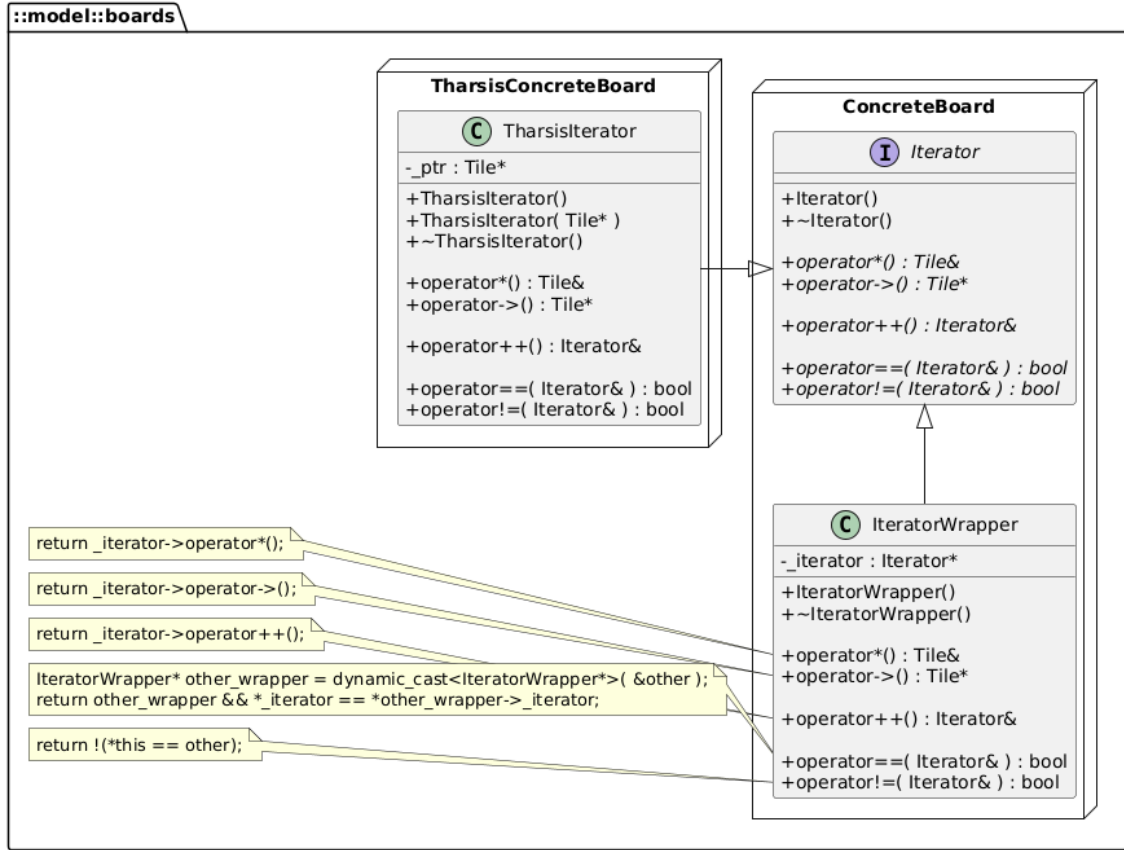


Figure 20: Class diagram of the iterators.

There are advantages and disadvantages of subclassing **IteratorWrapper** from **Iterator**. On one hand, it enforces any change in the **Iterator** interface to be updated in **IteratorWrapper**. On the other hand, someone may accidentally assign an instance of **IteratorWrapper** to an **Iterator** variable, and object slicing happens. However, this has a really low probability, as these classes are enclosed in the **ConcreteBoard** class.

3.1.2 Decks and cards

The `::model::decks` namespace contains classes implementing project cards, different sets of cards, and a **Deck** class.

In the board game, each card has a unique name, and an ID by design. I imported these into the implementation, creating an enumerated type, **CardID**, where identifiers bear the name of the card, and have their values set to their respective ID.

Cards

The game features three types of cards:

- **Event cards** (orange) only have effects when they are played, and after that they are turned face down. Their tags do not get counted for requirements. Victory points still count at the end of the game.
- **Automated cards** (green) also only have effects when they are played, but they stay face up and their tags are in effect for the rest of the game.
- **Active card** (blue) are similar to automated cards, but they either have an *effect* or an *action*.
 - *Effect cards* have ongoing effects, that trigger automatically on specific events (e.g. when the player places a city they gain some resources, or all cards cost one less).
 - *Action cards* have an action, that the player can choose to perform once each generation. They may have a cost needed to perform said action.

There can be multiple different actions on one card. In this case, only one of them can be used in each generation.

Action cards have an associated enumerated type **Availability**, which represents their state:

- usable
- not usable
- used this generation

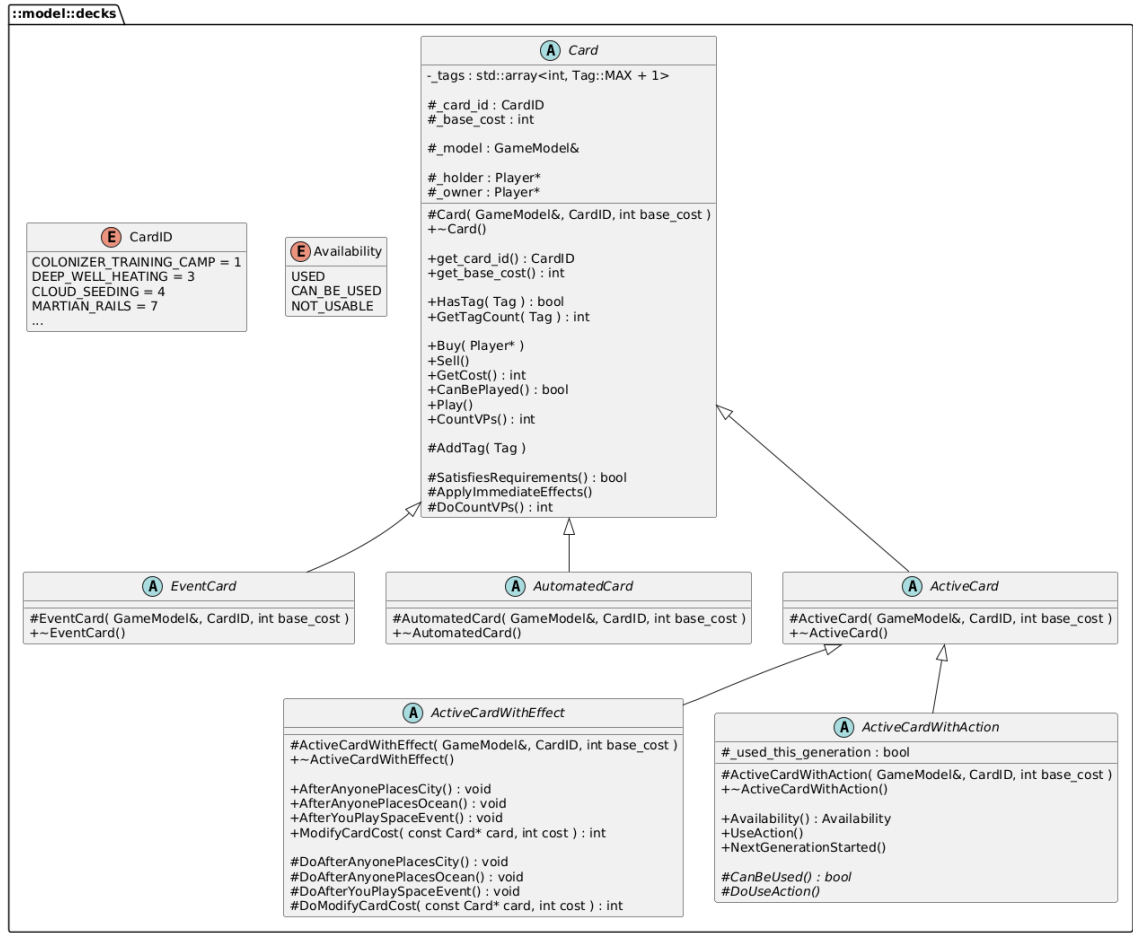


Figure 21: Class diagram of the cards.

The tag system is implemented in the **Card** parent class. The private `_tags` member holds this information, and derived concrete classes should utilize The **AddTag** protected method in their constructors to configure the correct amount of tags.

The *template method design pattern* is used a lot in these classes. **Card**'s other protected methods are all virtual, and they are used by the methods listed in the next paragraphs.

A card has some kind of state and state-dependent logic. These are the three states cards can have:

- The card is in the deck if $_holder = nullptr \wedge _owner = nullptr$
- The card is in the player's hand if $_holder \neq nullptr \wedge _owner = nullptr$
- The card was played by the player if $_holder \neq nullptr \wedge _owner \neq nullptr$

The following methods have state-specific behaviour: **Buy**, **Sell**, **GetCost**, **CanBePlayed**, **Play** and **CountVPs**. **Buy** just sets `_holder` to the received parameter, and **Sell** sets it back to `null`. This means that the credits received or paid have to be handled elsewhere. **GetCost** requires a holder as this calculates the cost with

cost reductions from the holder's effect cards. **CanBePlayed** also requires a holder, and uses the virtual **SatisfiesRequirements** method. **CountVPs** requires an owner, as only then can victory points be awarded.

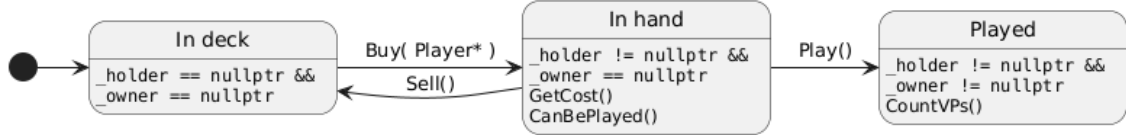


Figure 22: State diagram of the **Card** class.

Playing a card is a more complicated process. The **Player** class holding the **Card** instance in its hand has a **PlayCard** method, which handles most of the logic. Resources cannot be deducted in the **Card** class, since the **Player** might pay with Steel or Titanium resources if the card is of Building or Space type. In this case, **DoPlayCard** gets passed as a function pointer to be called after payment confirmation.

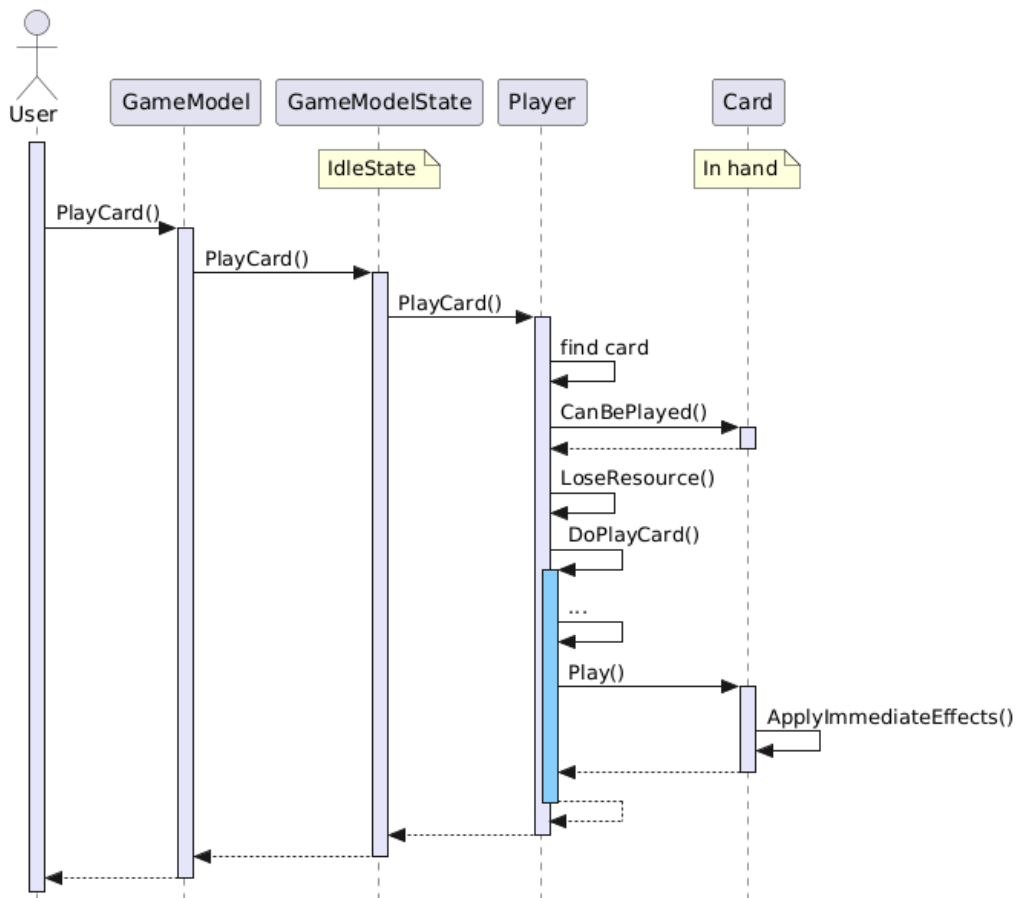


Figure 23: Sequence diagram of a card being played.

The concrete cards are contained in the `::model::decks::cards` namespace as to not pollute the `::model::decks` namespace. Adding a new card can simply be done by creating a new class, inheriting from one of the four types of cards (**EventCard**, **AutomatedCard**, **ActiveCardWithEffect**, **ActiveCardWithAction**), calling the parent constructor with the appropriate parameters, calling `AddTag` in it for all the card's tags and overriding the appropriate methods. In the `::model::decks::cards` namespace there is a header file located which includes all the cards. This way, when dealing with the concrete card classes one does not have to include them separately by hand.

The deck and deck providers

In the course of the game, players can draw cards from a pile of cards - the deck - and sell - discard - cards they decide to. To enable multiple card sets for the game, I created a **DeckProvider** abstract class, which has one method, `GenerateDeck`, which returns a freshly created list of cards. This method has to be overridden in derived classes, returning a different set of cards for each subclass. These classes can even implement the *singleton design pattern*.

An instance of the **DeckProvider** class can be injected into the **Deck** class. **Deck** initializes itself with the cards that `GenerateDeck` returns, shuffling them. **Cards** can be drawn from **Deck**, and discarded cards get stored in another member. When the deck is empty at card draw, discarded cards get shuffled and put back into the deck.

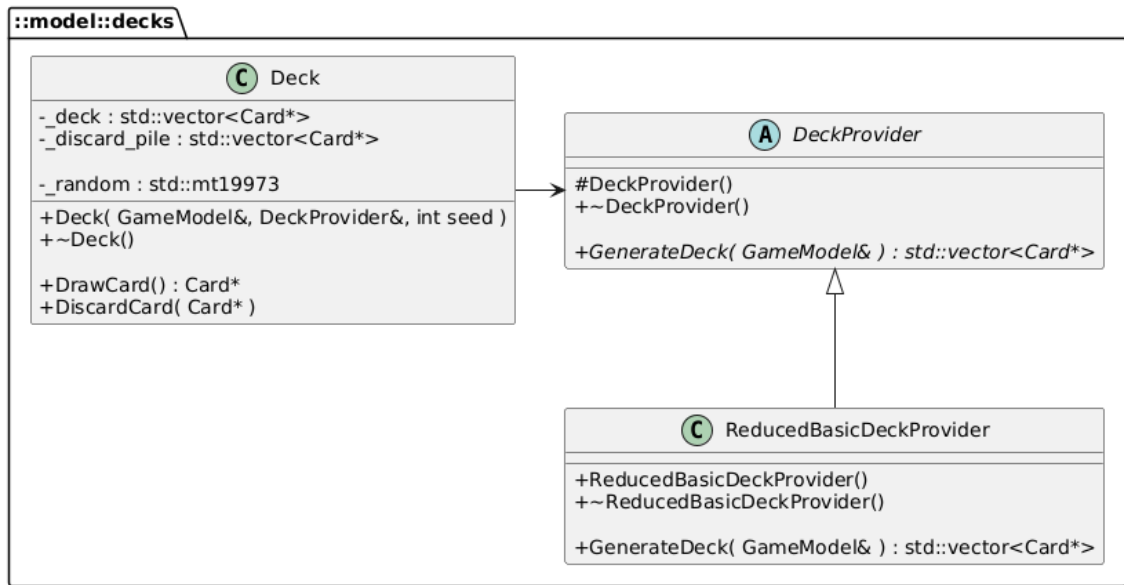


Figure 24: Class diagram of the decks.

3.1.3 `::model`

This section contains classes directly in the namespace `::model`. There are two enumerated values: **Tag** and **Resource**.

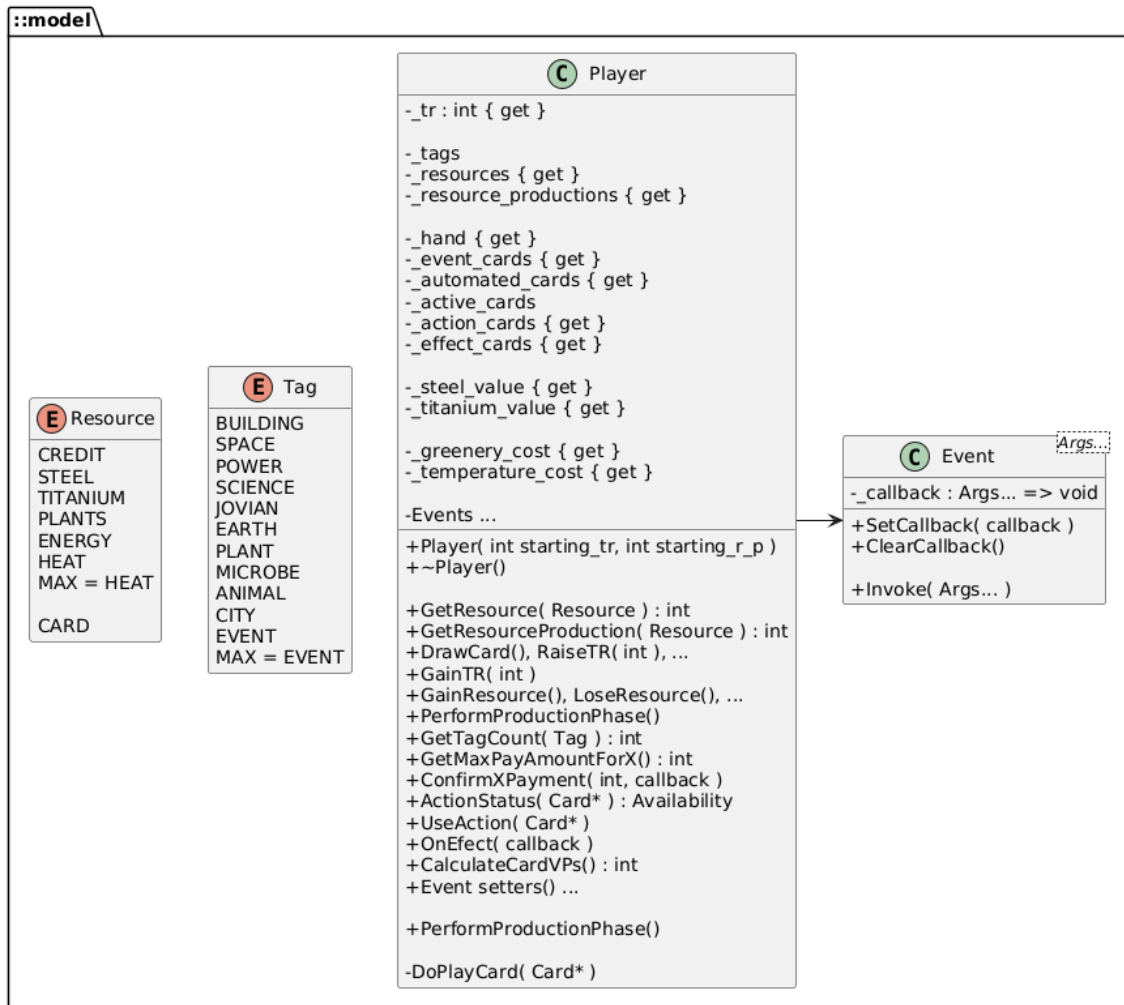


Figure 25: Class diagram of utilities in *::model*.

Event

The model contains an **Event** class for event-driven programming. The board, the **Player** and the **GameModel** classes all feature events. However, this **Event** class is not really an event. It only stores one function, so at any time a maximum of one function can be subscribed to the event.

Player

The player class stores data about a player. Resources, resource productions are stored in an array indexed by the target resource's underlying enumerated value. There are lots of events to make the program centralized: every time a player gains or does something, it comes in here. Methods usually have two kinds: the first invokes an event, notifying the **GameModel** about the requested change, and the latter one confirms and actually carries out the change (e.g. **RaiseTR** invokes an event, and

GainTR modifies the `_tr` member in **Player**). There are also methods for handling card ownership and play.

Core Model

The game leans heavily toward a state-centric architecture. The model was designed with this in mind from the start, and I am using the *state design pattern*. As before, it is still important to make a construct that allows new game modes and variants to be implemented easily and following the SOLID design principles. I identified the following states during planning:

- **Research state:** Corresponds to the research phase in the game. The player receives four cards, and can buy as many of them as they want, each costing 3 Credits.
- **Idle state:** Corresponds to the action phase in the game. The player can play cards, sell cards, use actions, use standard project, convert resources and end their turn.
- **Placement confirmation state:** Occurs whenever the player is placing a tile. A position is required for placing the tile, except when the card *Noctis City* is played and the game board has a reserved tile for it. In this case the city gets placed automatically.
- **Payment confirmation state:** Occurs whenever the player chooses to perform an action that requires payment that can be paid for by Steel or Titanium resources (card playing, specific actions on cards) and he has at least one of the correct resource type. If they have at least one resource, this state will occur even if they can only afford the payment by spending all their Credits and resources, to give feedback to the player about the fact they are spending their resources.
- **Post last generation state:** After the last generation, players have the option to convert their Plant resources to greeneries as normal. This is the only action they can perform here. Placement bonuses still trigger. After this phase, the game is over.

-
- **Post last generation placement confirmation state:** Placing greeneries in post last generation state transition into this state for clarity instead of the normal placement confirmation state.
 - **Game over state:** After the greenery converting phase, the game goes to this state. No actions are available here.

GameModel is an abstract class, that implements most of the features required for a usual game of Terraforming Mars. The base states inheriting from **GameModelState** were written for this abstract **GameModel**, which has virtual **CreateStateVariant** methods. Whenever a state change is about to happen and a new state instance has to be created, they get constructed through these methods. Different game variants can then inherit from **GameModel**, and if they need they can also create specialized state classes inheriting from the different **GameModelState** classes (or even from **GameModelState** itself). If they do, they need to override the appropriate **CreateStateVariant** method so that the model can create the correct state instance for the variant.

Let's go over this with the solo variant. **SoloGameModel** inherits from **GameModel**, and overrides some methods: the abstract **CreatePlayer** method, since starting terraform rating and resource productions varies between game modes. Turn ending logic is also different in solo games, so I created **SoloIdleState** and **SoloPostLastGeneration**, both inheriting from their respective parent classes.

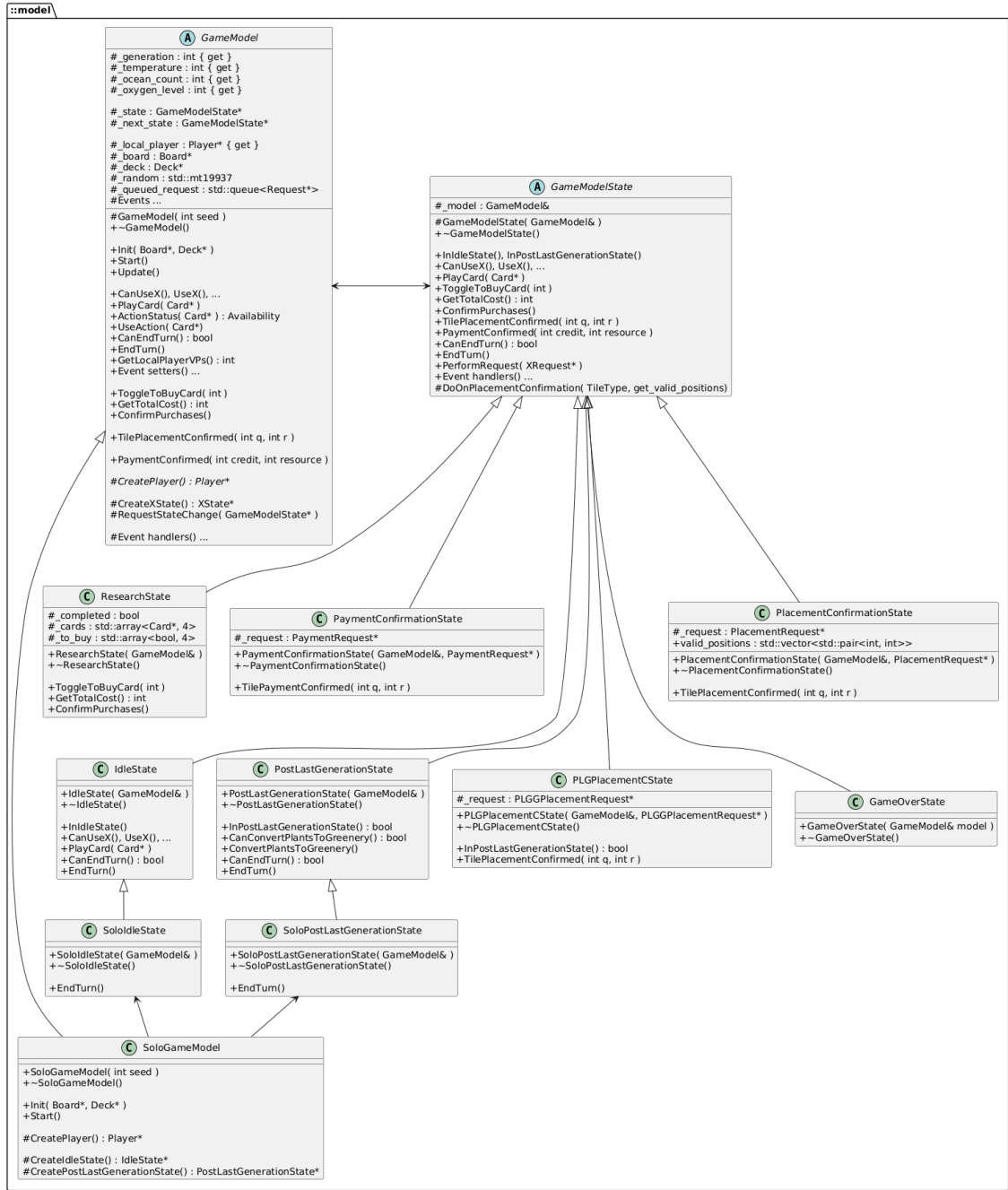


Figure 26: Approximate class diagram of the game model.

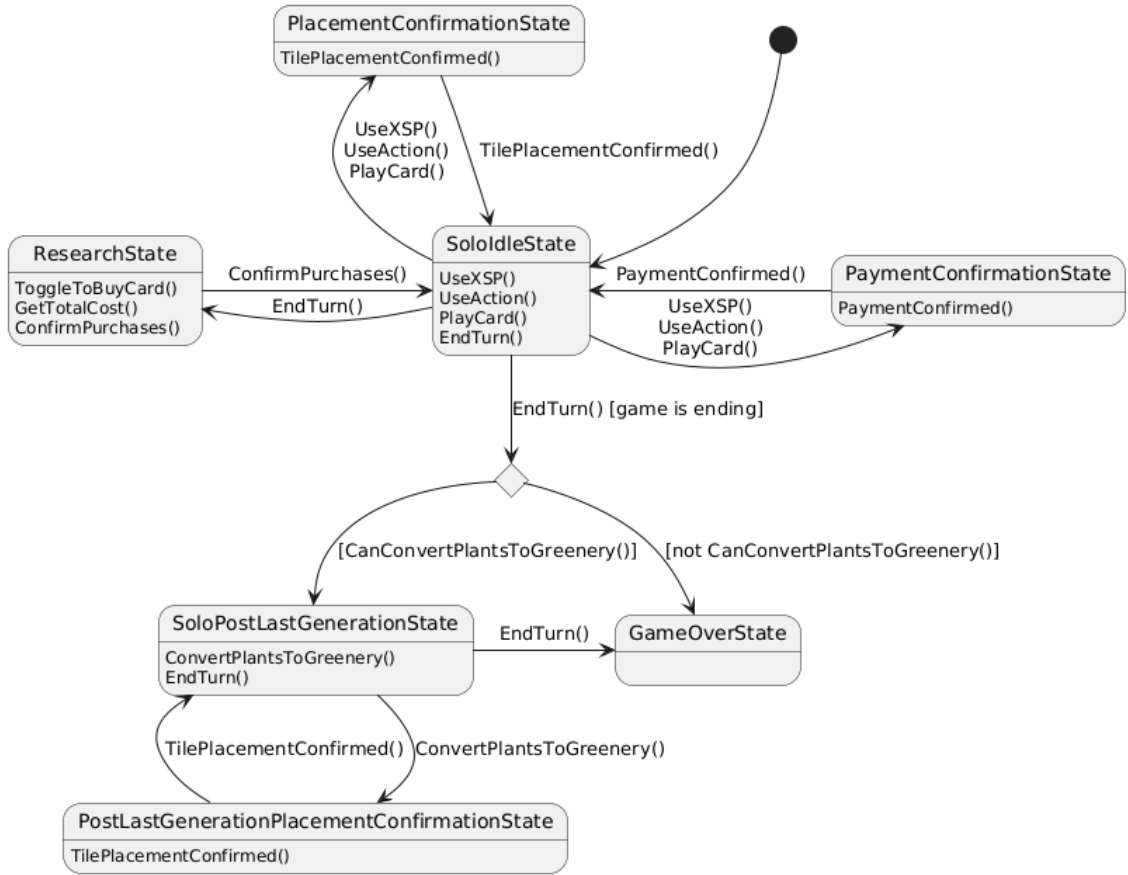


Figure 27: State diagram of the model for the solo variant.

Because only one tile can be placed at a time, but one action can result in multiple tile placements, there has to be some sort of mechanism that allows this behavior. For this, I created a **Request** base class in **GameModel**, and queue that stores pending request. In the idle state, if there is a request, it gets performed. In placement and payment confirmation state, after confirmation was received and there are requests, instead of going back to the idle state, the next request gets performed.

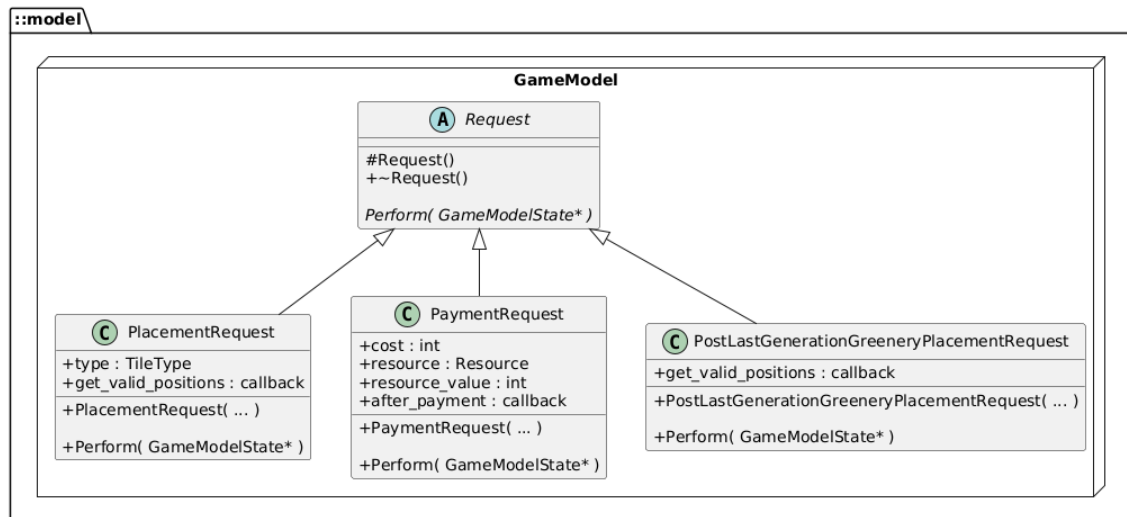


Figure 28: Class diagram of the requests.

For performing the next request, I use the *visitor design pattern*. After popping the next request from the queue, the abstract method **Perform** gets called, in which every subclass calls the state's **PerformRequest** overloaded method with *this* as argument.

There is a flaw in this design: you cannot directly change the state from a state method, as it can result in dangling references in the call stack. This cannot be solved with `std::shared_ptr`, as its reference counter does not increase on method calls on the held object.

To solve this, I created the **RequestStateChange** method, which sets `_next_state` to the parameter, and created an **Update** method which sets the state to the new one. This means that the model has an update method that has to be called frequently, in a render loop probably, which is far from optimal. Resolving this issue is important in the future.

ezt már nem
tudtam itt
értelmesen
megfogalmazni

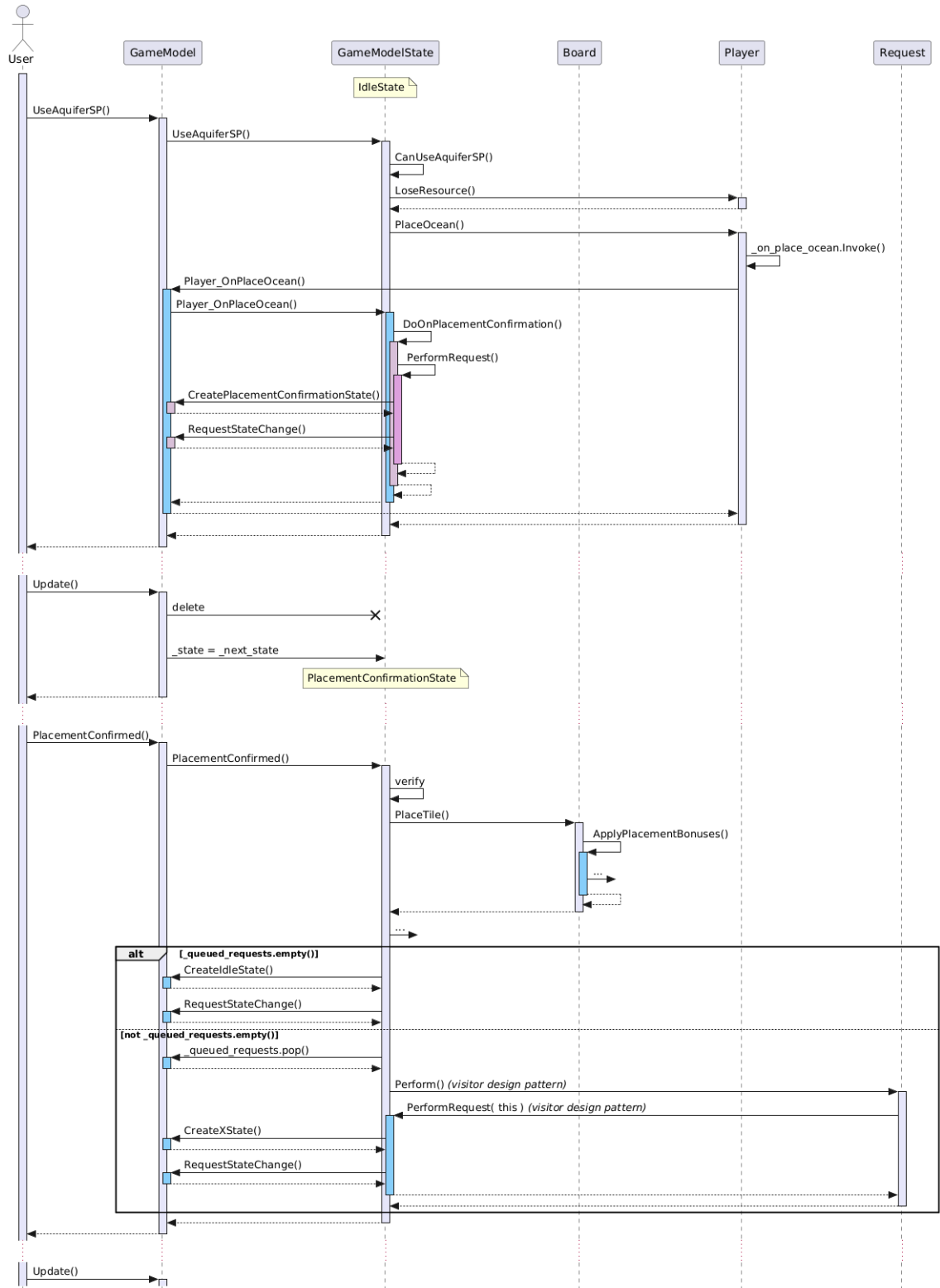


Figure 29: Sequence diagram of an example for a tile placement.

3.2 View

Aside from *main.cpp*, every class is wrapped in the *::view* namespace.

As in *Model*, I have created a *constants.hpp* file in this project too for constant values, like positions, animation durations and more.

3.2.1 Utilities

GL utils

Some code was borrowed from the computer graphics team, mainly *gl_utils.**, *camera.* and *spherical_camera_manipulator..* I created an interface for camera manipulators and implemented a bobbing camera manipulator that "follows" the mouse on the game screen.

Text rendering

The application needed some way to render text to the screen. For this, I implemented the **TextRenderer** static class. The base structure comes from [LearnOpenGL](#). I have tweaked it to adapt it to my usage of OpenGL, and added some functionality, like centered text rendering. Note that this class can only render one line texts. At this point, only the first 128 characters of the ASCII character set.

Animatable

For user experience and enjoyment, it is important that changes are smoothly animated, and not just abruptly appear on-screen. OpenGL provides no tools for this, so I had to implement tools for this from scratch. Luckily, GLM comes with a variety of easing functions that we can use for our animations.

The *animatable.hpp* header contains two templated data structures. **AnimationParams** is a simple template struct, containing the start and end value of an animation and the duration and elapsed time in seconds. **Animatable** is a template class which provides richer features for working with events.

At all times **Animatable** stores the current value it's holding. The type of *_value* is **T** which is the template parameter of **Animatable**. It can be accessed with the indirection operator and the structure dereference operator (*** and *->*). The value can be set at any time with the **Set** method, but this clears any ongoing animation. The class also contains an easing function, that can be set at any point.

[Its default value is linear interpolation.](#)

There can either be an ongoing animation effecting this value, or not. This is expressed with the member `_animation` of type `std::optional<AnimationParams<T>>`. When this optional contains a value, an animation is ongoing.

An animation can be set in two ways. `SetAnim` takes a start and end value and a duration. It clears any ongoing animation, and starts a new animation with the supplied parameters. `UpdateAnim` on the other hand takes in the end value only, a duration and a boolean value `force_time` indicating to whether or not force duration override.

- If there is no animation ongoing, a new animation is set, where the start is the current value of the **Animatable**, the end is the parameter of the method, and the duration also.
- If there is an ongoing animation, the end value is updated to the parameter of the method. If `force_time` is true, the elapsed value gets reset to zero, and the duration gets set to the duration parameter.

The class features an `Update` method that takes the delta time, and updates the value based on the animation and the easing function. If the duration is over, it sets the value to the end value and clears the animation.

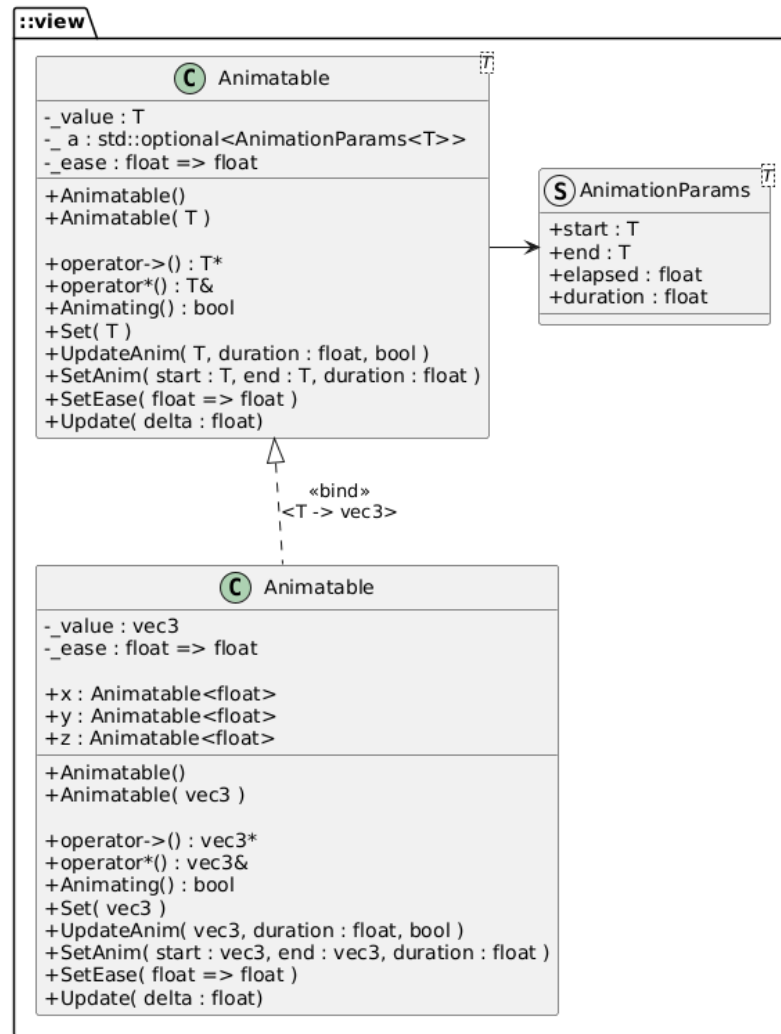


Figure 30: Class diagram of animation utilities.

Since in the application we deal with colors, positions and much more that consist of vector data, it was required that `glm::vecX` types could be animated too. I solved this with template specialization. On Figure30, the template specialization for `glm::vec3` is shown. An `Animatable<float>` is stored for each item of the vector. The base methods call their respective methods on each of the animatables. They are also public, so each one can be accessed and set independently. The value is also stored in an auxiliary `_value` member.

TileWrapper

This class wraps one field of the game board. Information for rendering is stored here.

CardWrapper

This class wraps a card. It features a state in the form of an enumerated type, which features the following values: `IDLE`, `HOVERED`, `DRAGGING`, `DRAWING` and `TO_HAND`. Another enumerated type **Visual** stores what kind of visual effect the card currently has. It can have the following values: `NONE`, `HIGHLIGHT`, `ACTION_HIGHLIGHT`, `SELL_HIGHLIGHT` and `FADED`.

Data for rendering is stored also, for example position, scale, and rotate. These are an instance of **Animatable**. The class implements methods to set all the easing functions at once.

3.2.2 Views

View is an abstract class that defines operations a view should or can implement: `Update`, `Render`, handling window, keyboard and mouse events.

Game

GameView has similar structure to **GameModel** and the same thought behind it for extensibility. It also features the *state design pattern*, but has slightly different states. There is an additional sell state, where the player can sell their cards, and there is no separate placement confirmation state for post last generation yet.

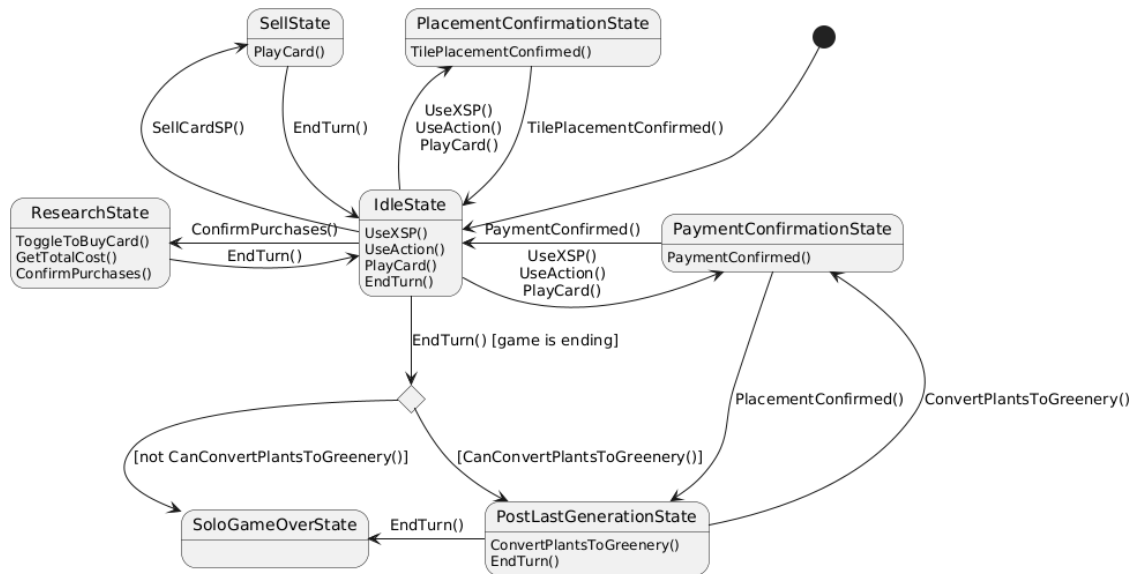


Figure 31: State diagram of the view of the solo variant.

SoloGameView inherits from **GameView** and has solo variant state specialization also in the form of **SoloGameOverState**, as the end of the game is simpler in the solo variant than in the base game.

Menu

MenuView implements **View**. This view renders the menu screen, with a title, new game and quit buttons, and if there is an ongoing game, a continue button. When the user selects the new game or the continue button, the view starts a transition to the game screen. For this, it invokes an event asking the Mars to be rendered. It signals the ending of the transition with another event invocation.

App

The **App** class holds together the application. It stores an instance of **MenuView** and optionally an instance of **GameView**. It features a `_current_view` member, which it updates and renders. But it is important to call **Resize** on every living **View** instance.

Builder

To create a **GameView** that is populated with its respective **GameModel** which also needs to be initialized with a **DeckProvider** and a **Board**, I used the *builder design pattern* to separate the construction of this complex object stack.

MenuView uses **Builder** to create a new **GameView** when new game is pressed. After that, it passes the created instance back to **App**.

main.cpp

The entry point of the application. It initializes subsystems, libraries and contexts. It creates an instance of the **App** class and initializes it. After these configurations, it runs the render loop, in which the following steps get performed:

- Processes every event from SDL. Most of them get passed down to **App** also.
- Call **Update** on **App**.
- Call **Render** on **App**.

-
- Call `RenderGUI` on **App**.
 - Finish rendering by swapping the framebuffers.

3.2.3 Animations

Using **Animatable**, we can create multiple animations. Since one action (like playing a card) can result in a lot of things happening (raising parameters, gaining resources, placing a tile all because of one card) I implemented an animation queue, where every animation has a set interval until the next animation cannot be started.

So **Animation** is an abstract class, that implements this behavior. The constructor takes an optional *lockout_time* duration parameter. The update method handles the decreasing of this member, and then calls the abstract method `DoUpdate`. The class also contains two optional callables, that get called when the animation starts and ends.

The **Render** abstract method is what renders the animation. This is implemented with the *visitor design pattern*: when **GameView** wants to render an animation, it calls this method, in which the children classes can call back the **RenderAnimation** with themselves as the parameter.

Concrete animations can then be derived from this class. For example, the **TextAnimation** class features a string to render, and its position, color and scale all wrapped in an instance of **Animatable**. `DoUpdate` calls `Update` on the animatables. Rendering is done with the current values from the animatables.// However, this class is pretty flexible, and can be used many ways. For example, **CardAnimation** only renders once, and it sets the respective **CardWrapper**'s animatable properties' animations.

In **GameView**, there is an animation queue with the pending animation, a vector with ongoing animations, and an `std::optional<Animation*>` which is the animation that currently locks, meaning no animation can be started. These are updated in every loop by the following steps:

- Every ongoing animation gets processed: If it is over, it gets removed from the vector and deleted, else it gets updated.
- If there is a locking animation: if it still has time from its lockout, it simply gets updated, else it gets transferred to the ongoing animations, and updated (if it is not over already in which case it just gets deleted).

-
- While there is no locking animation and the animation queue is not empty: the next animation gets popped from the queue. If it has a lockout duration, it gets assigned to be the next lockout animation, else it gets added to the ongoing animations.

There is also an implemented but unused feature: animations can be put in a collection with a delay. While the delay is not over, they do nothing. Once the delay is over, they get transferred to the ongoing animations. This feature doesn't support lockout.

3.2.4 Rendering

App

App contains the camera for the application. During rendering, the first thing rendered is the skybox. This always gets rendered, so **App** handles this. The skybox is slowly rotated to achieve a deeper space feeling.

Mouse picking

By default, OpenGL provides three framebuffers that store render data.

- The color buffer has three float values for every pixel, ranging from 0 to 1. This is the frame being rendered, each of the three values corresponding to one of the tree colors: red, green and blue.
- The depth buffer usually has 24 bits for every pixel. It is used to detect if the pixel being drawn is behind the already rendered pixel.
- The stencil buffer usually has 8 bits, so its values can range from 0 to 255. It can be used for various different rendering tricks and after-effects.

I used the stencil buffer for mouse picking. Each clickable element has an associated ID, and on a mouse down, mouse up or mouse move event we can poll the current value of the corresponding pixel of the stencil buffer. This way, we can identify which element was clicked or hovered over.

What ID to render can be set with the **SetStencilRef** method, but after rendering, it has to be set back to its original value, by calling **SetStencilRef** again without a parameter.

Menu

On the menu screen, the title and three buttons need to be rendered. These are pre-made *.png* files, as during the transition they need to be rendered 3D. The three buttons can be hovered and clicked.

Game

megírni a véglegesre

First the game board is rendered. This is done per tile.

Before rendering the HUD, the depth buffer is cleared, as everything here has to overlay what has been rendered so far. First the generation counter, menu button, end button, standard projects, global parameters and resources get rendered. After that, any state-specific rendering is carried out by calling **Render** on the current state. After that, the card panels get rendered, then the hand, and finally the animations.

HUD rendering is basically 2D rendering, so it does not use the camera. Positions and sizes are calculated dynamically and relative to the window and window size.

Cards and resources get their textures from sprite sheets. Card textures are derived from their **CardID**'s underlying value. Vertical and horizontal indices and strides are calculated in code, then get passed to the shaders with uniforms.

Cards can have a colored highlight around them. This was done in the fragment shader. If the alpha value is big, a highlight will be drawn. The alpha value of the highlight is based on the distance from the edge. The threshold that represents the highlight being a solid color moves in and out to create a pulsating effect.

4 Testing

Part IV

Conclusion

Acknowledgements

Appendix A

Simulation results

List of Figures

1	The menu screen of the application with an ongoing game.	7
2	Game screen in the action state.	8
3	Payment confirmation window.	9
4	Placement confirmation. On the top, the tile to be placed is shown, and valid locations have a white border.	10
5	Research phase.	11
6	End of a game.	12
7	Standard projects and resource conversions.	13
8	Global parameters and terraform rating.	14
9	Resources. Notice how Credit generation is the only one that can go into negative, down to -5.	14
10	Approximate location of the above-mentioned invisible line.	15
12	Action card panel. <i>Equatorial Magnetizer</i> and <i>Martian Rails</i> have been used this generation. While the others are still usable, <i>Ore Processor</i> cannot be used at this moment (the player does not have the required cost of 4 Energy).	17
13	Use case diagram of the menu screen.	21
14	Use case diagram of the game screen.	22
15	Detailed use cases of the Action state.	22
16	Package diagram of the application	23
17	The axial coordinate system. Notice the third coordinate in a lighter color.	25
18	Class diagram of the <code>::model::boards</code> namespace.	27
19	Example of memory layout when storing a board in a 2D array. . . .	28
20	Class diagram of the iterators.	29
21	Class diagram of the cards.	31
22	State diagram of the Card class.	32

23	Sequence diagram of a card being played.	32
24	Class diagram of the decks.	34
25	Class diagram of utilities in <i>::model</i>	35
26	Approximate class diagram of the game model.	38
27	State diagram of the model fo the solo variant.	39
28	Class diagram of the requests.	40
29	Sequence diagram of an example for a tile placement.	41
30	Class diagram of animation utilities.	44
31	State diagram of the view of the solo variant.	45

List of Tables

List of Algorithms

List of Codes

1	game_model.fwd.hpp	24
---	------------------------------	----